

ragent Tools Reference

A complete catalog of every tool available to ragent agents, organized by category. Each tool includes its name, description, use cases, parameter schema, the system instruction the model receives, and a worked example.

Scope: Tool names, schemas, and usage patterns. For TUI workflow see [docs/howtos/tutorial.md](#). For hiding/exposing tool families see [docs/howtos/tool-visibility.md](#). For team coordination see [docs/howtos/howto_teams.md](#).

Category Index

#	Category	Count	Visibility Switch
1	File Operations	18	always on
2	Shell	5	always on
3	Search	1	always on
4	Web	3	always on
5	Browser Automation	1	browser
6	MasterFetch	6	masterfetch
7	Code Intelligence	10	codeindex
8	Memory	5	always on
9	Git	18	always on
10	GitHub	11	github
11	GitLab	19	gitlab
12	Office & PDF	8	office
13	Teams	20	teams
14	Sub-Agents	5	agents
15	Planning	2	plan
16	Spec Management	5	always on
17	Task Management	4	always on
18	Scheduling	5	always on
19	Initiatives	1	always on
20	MCP	1	always on
21	Skills	1	always on

#	Category	Count	Visibility Switch
22	Interactive	4	always on
23	Finance	8	finance
24	Communications	2	always on

Switches default `off` for `github`, `gitlab`, `teams`, `agents`, `plan`, `office`; the rest default `on`. See <docs/howtos/tool-visibility.md>.

1. File Operations

Tool	Description
<code>read</code>	Read file contents with optional line range.
<code>write</code>	Create or overwrite a file.
<code>create</code>	Create a new file (preferred for new files).
<code>edit</code>	Replace one exact text occurrence in a file.
<code>multi_edit</code>	Apply multiple surgical edits atomically across files.
<code>multiedit</code>	Deprecated alias for <code>multi_edit</code> .
<code>apply_patch</code>	Apply a Codex-style patch (<code>*** Begin Patch</code>).
<code>patch</code>	Apply a unified diff patch.
<code>append_to_file</code>	Append text to the end of a file.
<code>rm</code>	Delete a single file.
<code>move_file</code>	Move or rename a file/directory.
<code>copy_file</code>	Copy a file to a new location.
<code>make_directory</code>	Create a directory (<code>mkdir -p</code>).
<code>file_info</code>	Return metadata for a file or directory.
<code>diff_files</code>	Unified diff between two files or strings.
<code>glob</code>	Find files matching a glob pattern.
<code>list</code>	List directory contents in a tree.
<code>update_file</code>	Alias for <code>write</code> (overwrite existing file).

Use cases: scaffolding projects, editing source, comparing files.

Schema (`edit`): `{"file_path":"string","old_string":"string","new_string":"string"}`

System instruction: "Use `edit` for single surgical replacements. `old_string` must match exactly once. Use `multi_edit` for changes across multiple files."

Example:

```
edit file_path="src/main.rs" old_string="fn main() {" new_string="fn main()  
-> Result<()> {"
```

2. Shell

Tool	Description
<code>bash</code>	Execute a shell command (7-layer security).
<code>bash_reset</code>	Reset persistent shell state.
<code>bg</code>	Manage background shell tasks (spawn, list, wait, cancel).
<code>run_code</code>	Alias for <code>bash</code> accepting <code>code</code> as <code>command</code> .
<code>open</code>	Open a file/folder/URL via the desktop handler.

Use cases: running builds, tests, git operations, long-running tasks.

Schema (`bash`): `{"command": "string", "timeout": "int?"}`

System instruction: "For simple commands use `bash` with the `command` parameter immediately. Do not describe what you will run."

Example:

```
bash command="cargo test -- --nocapture" timeout=600
```

3. Search

Tool	Description
<code>grep</code>	Search file contents for a regex pattern (ripgrep).

Use cases: finding TODOs, searching comments, non-symbol text patterns.

Schema: `{"pattern": "string", "path": "string?", "include": "string?"}`

System instruction: "Use `grep` for arbitrary text and pattern matching. Use `codeindex` tools for code symbol queries."

Example:

```
grep pattern="TODO" path="src" include="*.rs"
```

4. Web

Tool	Description
<code>webfetch</code>	Fetch a URL via HTTP GET; HTML converted to text.
<code>websearch</code>	Web search returning titles, URLs, snippets.
<code>http_request</code>	Full HTTP method/headers/body control.

Use cases: fetching documentation, testing APIs, simple web scraping.

Example:

```
webfetch url="https://rust-lang.org"
http_request url="https://api.github.com/repos/rust-lang/rust" method="GET"
```

5. Browser Automation

Tool	Description
<code>browser</code>	Chrome DevTools Protocol automation (open, click, type, screenshot, etc.).

Use cases: interacting with dynamic web pages, form filling, screenshots.

Schema: `{"action": "open|click|type|screenshot|...", "url": "string?"}`

Example:

```
browser action="open" url="https://example.com"
browser action="screenshot"
```

6. MasterFetch

Tool	Description
<code>mf_fetch</code>	Rich URL/PDF extraction with envelope signals.
<code>mf_search</code>	Keyless multi-engine web search.
<code>mf_crawl</code>	Best-first same-domain crawl.
<code>mf_cache_clear</code>	Clear the content cache.
<code>mf_screenshot</code>	Capture a page as a screenshot.
<code>mf_version</code>	Return integration version info.

Use cases: extracting article content, searching multiple engines, crawling a domain.

Example:

```
mf_fetch url="https://example.com" format="markdown"
mf_search query="rust async patterns" max_results=10
```

7. Code Intelligence

Tool	Description
<code>codeindex_search</code>	Search symbols by name/keyword.
<code>codeindex_symbols</code>	Query symbols with kind/file filters.
<code>codeindex_references</code>	Find all references to a symbol.
<code>codeindex_dependencies</code>	Query file import/dependent edges.
<code>codeindex_status</code>	Show index status and statistics.
<code>codeindex_reindex</code>	Trigger a full re-index.
<code>codeindex_explain</code>	Explain a graph node and its edges.
<code>codeindex_path</code>	Shortest path between two symbols.
<code>codeindex_communities</code>	Community detection over the graph.
<code>codeindex_godnodes</code>	Top-N most-connected symbols.

Use cases: finding function definitions, tracing callers, dependency analysis, code graph exploration.

System instruction: "MUST use codeindex instead of `grep` for code symbol queries. Use `grep` only for arbitrary text patterns."

Example:

```
codeindex_search query="parse_config" kind="function"
codeindex_references symbol="parse_config"
```

8. Memory

Tool	Description
<code>memory_store</code>	Store a structured memory (category, tags, confidence).
<code>memory_recall</code>	Full-text search of structured memories.

Tool	Description
<code>memory_forget</code>	Delete memories by ID or filter.
<code>conversation_search</code>	Search current session conversation history.
<code>session_search</code>	Search across all past sessions.

Schema (`memory_store`):

```
{"content": "string", "category": "fact|pattern|insight", "tags": [...], "confidence": 0.7}
```

Example:

```
memory_store content="We use anyhow::Result for errors"
category="preference" confidence=0.9
memory_recall query="error handling"
```

9. Git

Tool	Description
<code>git_add</code>	Stage files for commit.
<code>git_branch</code>	List local and remote branches.
<code>git_checkout</code>	Switch branch or restore files.
<code>git_cherry_pick</code>	Apply changes from existing commits.
<code>git_clone</code>	Clone a repository.
<code>git_commit</code>	Create a commit from staged changes.
<code>git_diff</code>	Show working tree, staged, or commit diff.
<code>git_fetch</code>	Fetch from remote without merging.
<code>git_log</code>	Show commit history.
<code>git_merge</code>	Merge another branch.
<code>git_pull</code>	Fetch and integrate from remote.
<code>git_push</code>	Push branches and tags to remote.
<code>git_remote</code>	List, add, remove, update remotes.
<code>git_reset</code>	Unstage or reset to a commit.
<code>git_show</code>	Show commit, tag, or object details.
<code>git_stash</code>	Stash and unstash changes.

Tool	Description
<code>git_status</code>	Show working tree status.
<code>git_tag</code>	List, create, show, or delete tags.

Example:

```
git_status
git_add paths=["src/main.rs"]
git_commit message="Add greet subcommand"
git_push
```

10. GitHub

Tool	Description
<code>github_list_issues</code>	List issues (open/closed/all).
<code>github_get_issue</code>	Get full issue details.
<code>github_create_issue</code>	Create a new issue.
<code>github_comment_issue</code>	Comment on an issue.
<code>github_close_issue</code>	Close an issue.
<code>github_list_prs</code>	List pull requests.
<code>github_get_pr</code>	Get PR details.
<code>github_create_pr</code>	Create a new pull request.
<code>github_merge_pr</code>	Merge an open PR.
<code>github_review_pr</code>	Submit a PR review.
<code>github_get_actions</code>	List recent Actions workflow runs.

Requires `/github login` first. See `docs/howtos/tutorial.md` Section 7.

Example:

```
github_create_pr title="Add greet subcommand" base="main" body="Implements greet"
github_merge_pr number=42 method="squash"
```

11. GitLab

Tool	Description
<code>gitlab_list_issues</code>	List issues.
<code>gitlab_get_issue</code>	Get issue details.
<code>gitlab_create_issue</code>	Create an issue.
<code>gitlab_comment_issue</code>	Comment on an issue.
<code>gitlab_close_issue</code>	Close an issue.
<code>gitlab_list_mrs</code>	List merge requests.
<code>gitlab_get_mr</code>	Get MR details.
<code>gitlab_create_mr</code>	Create a merge request.
<code>gitlab_merge_mr</code>	Merge an MR.
<code>gitlab_approve_mr</code>	Approve an MR.
<code>gitlab_list_pipelines</code>	List pipelines.
<code>gitlab_get_pipeline</code>	Get pipeline details.
<code>gitlab_list_jobs</code>	List jobs in a pipeline.
<code>gitlab_get_job</code>	Get job details.
<code>gitlab_get_job_log</code>	Get job log output.
<code>gitlab_retry_job</code>	Retry a failed job.
<code>gitlab_cancel_job</code>	Cancel a running job.
<code>gitlab_retry_pipeline</code>	Retry a pipeline.
<code>gitlab_cancel_pipeline</code>	Cancel a pipeline.

Requires `/gitlab setup` authentication.

12. Office & PDF

Tool	Description
<code>office_read</code>	Read DOCX/XLSX/PPTX content.
<code>office_write</code>	Write Office documents.
<code>office_info</code>	Return Office document metadata.
<code>libre_read</code>	Read ODT/ODS/ODP content.
<code>libre_write</code>	Write LibreOffice documents.
<code>libre_info</code>	Return LibreOffice document metadata.

Tool	Description
pdf_read	Extract text from a PDF.
pdf_write	Write a PDF document.

Example:

```
office_read path="report.docx"  
pdf_read path="spec.pdf"
```

13. Teams

Tool	Description
team_create	Create a named team from a blueprint.
team_spawn	Spawn a teammate for a scoped task.
team_message	Direct message a teammate or lead.
team_broadcast	Message all active teammates.
team_read_messages	Check mailbox for unread messages.
team_status	Team and task status summary.
team_task_list	List all team tasks.
team_task_create	Lead adds a shared task.
team_task_claim	Teammate claims the next available task.
team_task_complete	Mark a claimed task done.
team_assign_task	Lead assigns a task to a teammate.
team_submit_plan	Teammate submits a plan to the lead.
team_approve_plan	Lead approves/rejects a plan.
team_wait	Block until teammates finish.
team_idle	Teammate signals no more work.
team_shutdown_teammate	Lead requests teammate shutdown.
team_shutdown_ack	Teammate acks shutdown and exits.
team_cleanup	Lead tears down the team.
team_memory_read	Read team memory bucket.
team_memory_write	Write team memory bucket.

See [docs/howtos/howto_teams.md](#) for the full team manual.

14. Sub-Agents

Tool	Description
<code>new_agent</code>	Spawn a sub-agent (blocking or background).
<code>cancel_agent</code>	Cancel a running background sub-agent.
<code>list_agents</code>	List sub-agent tasks for the session.
<code>wait_agents</code>	Block until background tasks complete.
<code>agent_complete</code>	Terminal signal: the autonomous task is done.

System instruction: "Prefer sub-agents over doing work yourself. Use `background: true` when spawning more than one."

Example:

```
new_agent agent="explore" task="Find all usages of EventBus in src/"
background=true
wait_agents
```

15. Planning

Tool	Description
<code>plan_enter</code>	Delegate to the plan agent for read-only analysis.
<code>plan_exit</code>	Exit plan mode.

16. Spec Management

Tool	Description
<code>spec_read</code>	Read a specification by ID.
<code>spec_list</code>	List all specifications.
<code>spec_search</code>	Search specifications by keyword.
<code>spec_task_update</code>	Update a task's status within a spec.
<code>spec_coverage</code>	Generate a requirement coverage report.

See [docs/howtos/spec.md](#) for the full spec workflow.

17. Task Management

Tool	Description
<code>task_create</code>	Create a session-scoped task.
<code>task_update</code>	Update task status, subject, or dependencies.
<code>task_get</code>	Retrieve a single task by ID.
<code>task_list</code>	List all session tasks.

Example:

```
task_create subject="Fix config loader" description="Reproduce panic and
fix root cause"
task_update task_id="task-001" status="in_progress"
```

18. Scheduling

Tool	Description
<code>cron_add</code>	Create a scheduled agent run.
<code>cron_remove</code>	Delete a scheduled event.
<code>cron_list</code>	List all scheduled events.
<code>cron_enable</code>	Enable a scheduled event.
<code>cron_disable</code>	Disable a scheduled event.

Schema (`cron_add`): `{"id":"string","agent":"general","schedule":"every 1h","prompt":"string"}`

19. Initiatives

Tool	Description
<code>initiative</code>	Manage durable initiatives with milestones.

Actions: `create`, `read`, `update`, `checkpoint`, `list`, `close`.

20. MCP

Tool	Description
<code>mcp_tool</code>	Bridge to external Model Context Protocol servers.

Tools are dynamically named `mcp_<server>_<tool>` at runtime after discovery via `/mcp discover`.

21. Skills

Tool	Description
<code>skill_manage</code>	List, read, load, or reload skill packs.

Actions: `list`, `read`, `load`, `reload`.

22. Interactive

Tool	Description
<code>ask_user</code>	Ask the user a question (with optional choices).
<code>think</code>	Record a short reasoning note.
<code>calculator</code>	Evaluate a mathematical expression.
<code>get_env</code>	Read environment variable values.

Example:

```
ask_user question="Which build profile?" options=["Debug","Release"]
calculator expression="2 ** 32"
get_env name="ANTHROPIC_API_KEY"
```

23. Finance

Tool	Description
<code>stock_quote</code>	Latest stock quote (price, volume, change).
<code>stock_history</code>	Historical OHLCV bars.
<code>stock_fundamentals</code>	Market cap, P/E, EPS, dividend yield, sector.
<code>stock_search</code>	Search ticker symbols by company name.
<code>stock_options</code>	Options chain (calls and puts).
<code>stock_recommendations</code>	Analyst recommendation trends.
<code>currency_rate</code>	Current exchange rate between two currencies.
<code>currency_history</code>	Historical exchange-rate OHLCV bars.

See `docs/howtos/finance.md` for configuration and provider details.

Example:

```
stock_quote symbol="AAPL"
stock_history symbol="MSFT" interval="1d" period="3mo"
currency_rate base="USD" quote="EUR"
```

24. Communications

Tool	Description
gmail	Search, read, draft, and send Gmail messages.
send_channel_message	Post to Telegram or Discord channels.

See [docs/howtos/communications.md](#) for OAuth2 setup and channel config.

Example:

```
gmail action="search" query="from:ci@example.com is:unread"
send_channel_message action="send" message="Build passed"
channel="telegram"
```

Related Documents

Document	Covers
docs/howtos/tutorial.md	End-to-end TUI workflow tutorial
docs/howtos/tool-visibility.md	Hiding and exposing tool families
docs/howtos/howto_teams.md	Multi-agent team coordination
docs/howtos/communications.md	Gmail and messaging channel tools
docs/howtos/finance.md	Stock and currency tools
docs/howtos/spec.md	Spec management and SDD workflow
docs/howtos/reverse.md	Repository reverse-engineering
docs/howtos/research.md	Research system and report synthesis
docs/howtos/custom-agents.md	Custom agent profiles and OASF schema